

08 bellman ford

Bellman-Ford Algorithm

1. What is it, and when is it used?

Bellman-Ford computes shortest paths from a single source to all other nodes in a weighted graph that may contain **negative edge weights**, and can also **detect negative-weight cycles**. It's used whenever Dijkstra's non-negative-weight assumption doesn't hold, or when you explicitly need to detect if a negative cycle exists (e.g., arbitrage detection).

2. Why is it used?

Dijkstra's greedy approach fails with negative edges because it finalizes a node's distance too early. Bellman-Ford instead **relaxes every edge, $V-1$ times**, guaranteeing that after $V-1$ rounds, all shortest paths (which have at most $V-1$ edges in a graph with no negative cycle) are correctly computed — regardless of edge sign. A bonus V -th round that still finds an improvement proves a negative cycle exists.

3. Where is it used?

- Currency arbitrage detection (negative cycle in a graph of exchange-rate log-weights)
- Shortest path in graphs with negative weights (e.g., cost that can be a refund/credit)
- Cheapest flights within K stops (natural fit — Bellman-Ford's round-based relaxation directly models "at most K edges")
- Network routing protocols (distance-vector routing, e.g., RIP, is essentially distributed Bellman-Ford)
- Detecting negative cycles in scheduling / constraint graphs (difference constraints systems)

4. How do you identify it's a Bellman-Ford problem?

Signal phrases (2-minute test): - "Edge weights can be negative" - "Detect if there is a negative cycle" - "Cheapest flights within K stops" (bounded number of edges) - "Currency exchange arbitrage" - "Shortest path" but weights represent something that can be a credit/refund (implying negative values) - Explicit mention of "at most K edges/stops" in a shortest path context

5. Can you identify it from constraints?

Constraint clue	Implication
Edge weights include negative values	Rules out Dijkstra → Bellman-Ford
Need to detect negative cycles	Bellman-Ford (extra V th relaxation round)
"At most K edges" bound given	Bellman-Ford's round-by-round relaxation naturally models this — run only K rounds
V, E small-to-medium (V up to ~ 5000 , E up to $\sim 10^4$ - 10^5)	$O(V \cdot E)$ is acceptable; if V, E are huge, Bellman-Ford may be too slow
Graph is DAG	Consider simpler $O(V+E)$ topological-sort-based shortest path instead

6. Types / Variants

1. **Classic Bellman-Ford** — relax all E edges, $V-1$ times; $O(V \cdot E)$.
2. **Bellman-Ford with Negative Cycle Detection** — run one extra round; if any edge still relaxes, a negative cycle exists (and is reachable from source).
3. **SPFA (Shortest Path Faster Algorithm)** — queue-based optimization of Bellman-Ford, often faster in

practice (but same worst case).

4. **Bounded-hop Bellman-Ford** — run only K rounds when the problem restricts path length to K edges (e.g., “cheapest flights within K stops”).
5. **Bellman-Ford for Difference Constraints** — modeling systems of inequalities $x_j - x_i \leq w$ as a graph and checking feasibility via negative cycle detection.

7. Rationale / Intuition

Any shortest simple path in a graph with V nodes uses **at most V−1 edges**. Bellman-Ford exploits this directly: relaxing every edge once guarantees all shortest paths using exactly 1 edge are found; relaxing all edges again guarantees paths using at most 2 edges are found; after V−1 total rounds, paths using up to V−1 edges (i.e., all possible simple paths) are correctly computed. If a valid negative cycle exists, distances would keep shrinking forever, so a **V-th round that still finds an improvement is proof of a negative cycle** reachable from the source (and able to reach the improved node).

8. Time & Space Complexity

Operation	Time	Space
Bellman-Ford (single source)	$O(V \cdot E)$	$O(V)$
Negative cycle detection (extra round)	$O(E)$	$O(V)$
Bounded K-hop Bellman-Ford	$O(K \cdot E)$	$O(V)$
SPFA (practical average case)	$O(k \cdot E)$, k small in practice	$O(V)$

9. Comparison with Other Patterns

Algorithm	Negative weights?	Detects negative cycle?	Time	Best when
Bellman-Ford	Yes	Yes	$O(V \cdot E)$	Negative edges present, or need cycle detection, or bounded-hop constraint
Dijkstra	No	No	$O((V+E) \log V)$	All weights non-negative — faster, prefer it when possible
Floyd-Warshall	Yes (no negative cycle)	Yes (diagonal check)	$O(V^3)$	All-pairs shortest paths, small dense V
BFS	N/A	N/A	$O(V+E)$	Unweighted graphs
SPFA	Yes	Yes	$O(k \cdot E)$ avg, $O(V \cdot E)$ worst	Same use case as Bellman-Ford, often faster in sparse practical graphs

Decision rule: **weights can be negative → Bellman-Ford** (unless you need all-pairs on a small graph, then Floyd-Warshall). Otherwise, Dijkstra is faster and should be preferred.

10. Reusable Java Templates

10.1 Classic Bellman-Ford with Negative Cycle Detection

```
import java.util.*;

class BellmanFord {
    // edges[i] = {u, v, weight}
    public int[] shortestPath(int n, int[][] edges, int src) {
        int[] dist = new int[n];
```

```

        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        // Relax all edges V-1 times
        for (int i = 0; i < n - 1; i++) {
            for (int[] edge : edges) {
                int u = edge[0], v = edge[1], w = edge[2];
                if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                }
            }
        }

        // One more round: if anything still relaxes, negative cycle exists
        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], w = edge[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                throw new IllegalStateException("Graph contains a negative-weight cycle");
            }
        }

        return dist;
    }
}

```

10.2 Bounded-Hop Bellman-Ford — Cheapest Flights Within K Stops

```

import java.util.*;

class CheapestFlightsKStops {
    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        // Allow at most k+1 edges (k stops = k+1 edges)
        for (int i = 0; i <= k; i++) {
            int[] temp = dist.clone(); // must use snapshot so we don't chain within the same round
            for (int[] flight : flights) {
                int u = flight[0], v = flight[1], w = flight[2];
                if (dist[u] != Integer.MAX_VALUE && dist[u] + w < temp[v]) {
                    temp[v] = dist[u] + w;
                }
            }
            dist = temp;
        }

        return dist[dst] == Integer.MAX_VALUE ? -1 : dist[dst];
    }
}

```

10.3 SPFA — Queue-Optimized Bellman-Ford

```

import java.util.*;

class SPFA {
    public int[] shortestPath(int n, List<int[]>[] adj, int src) {
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        boolean[] inQueue = new boolean[n];
        int[] relaxCount = new int[n]; // for negative cycle detection
        Deque<Integer> queue = new ArrayDeque<>();
        queue.offer(src);
    }
}

```

```

        inQueue[src] = true;

        while (!queue.isEmpty()) {
            int u = queue.poll();
            inQueue[u] = false;

            for (int[] edge : adj[u]) {
                int v = edge[0], w = edge[1];
                if (dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                    if (!inQueue[v]) {
                        queue.offer(v);
                        inQueue[v] = true;
                        if (++relaxCount[v] > n) {
                            throw new IllegalStateException("Negative cycle detected");
                        }
                    }
                }
            }
        }
        return dist;
    }
}

```

11. Quick Recognition Checklist (2-minute test)

- ☐ Edge weights can be negative?
- ☐ Need to detect a negative cycle?
- ☐ "At most K stops/edges" phrasing?
- ☐ V, E small/medium enough that $O(V \cdot E)$ fits time limits?

If 2+ checked → Bellman-Ford.